

Smart Food Court System

Introduction to FastAPI

- What is FastAPI?
- Core Technology Stack
- Automatic Interactive Documentation

What is FastAPI?

- Definition: FastAPI is a modern, fast (high-performance), web framework for building APIs with Python 3.8+ based on standard Python type hints.
- Key Pillars:
 - Fast: Very high performance, on par with NodeJS and Go (thanks to **Starlette** and Pydantic).
 - Fast to code: Increases the speed to develop features by about 200% to 300%.
 - Fewer bugs: **Reduces** about 40% of human (developer) induced errors.

Core Technology Stack

- FastAPI achieves its incredible speed and efficiency by standing on the shoulders of giants:
 - ASGI (**Asynchronous Server Gateway** Interface): Enables **true asynchronous** concurrency (handles **async/await** operations perfectly).
 - **Starlette**: Handles the core **web routing**, requests, and WebSocket capabilities.
 - **Pydantic**: Handles **data** validation, serialization, and automatically enforces data schemas using standard Python types.

Automatic Interactive Documentation

- One of FastAPI's best features is Zero-Configuration Documentation.
- Based on open standards (OpenAPI and JSON Schema).
- Two built-in UI options out of the box:
 - Swagger **UI (/docs)**: Allows interactive testing of endpoints directly from the browser.
 - ReDoc **(/redoc)**: Provides clean, professional, and well-structured API reference documentation.

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
```

```
async def read_root():
```

```
    return {"message": "Hello World"}
```

```
# To run the server:
```

```
# uvicorn main:app --reload
```

Introduction to Assignment

- **Project Title:** Smart Food Court System
- Objective: To help students master **fundamental asynchronous** operations in the asyncio library, including **create_task**, **gather**, **wait**, and **wait_for** using a relatable food court simulation.
- System Architecture:
 - Backend API (**foodcourt_api.py**): Simulates a kitchen with 3 shops running at different speeds (Chicken Rice: **0.8s**, Noodles: **1.5s**, Steak: **4.0s**).
 - Network Module (**food_utils.py**): A helper module providing an asynchronous function to send HTTP requests to the kitchen API.

Menu

- hainanese_chicken
- noodle
- steak

foodcourt_01_create_task.py

- Concept Taught: **Task Queueing** and Non-blocking Execution
- Key Mechanisms: **asyncio.create_task()**, **.done()**
- Scenario: Ordering a **single plate** of chicken rice by pushing it into the background Event Loop queue without blocking immediately.
- What this file teaches:
 - How to **wrap** a standard coroutine function into a Task object using **asyncio.create_task()** so that execution starts immediately in the background.
 - The usage of the **.done()** method to poll the current execution status (returning False right after creation since cooking takes time).
 - Using the await keyword later to pause execution and fetch the final response once the task is fully completed.

```
Tue Jul 14 20:18:54 2026 | --- [Task 1] Practice using create_task to queue an order ---
Tue Jul 14 20:18:54 2026 | Checking task status immediately: Is it done? = False
Tue Jul 14 20:18:55 2026 | System Response: {'status': 'READY_FOR_PICKUP', 'student_id': '65010001', 'shop': 'hainane
se_chicken', 'menu': 'Chicken Rice Mixed', 'cooking_seconds': 0.8, 'timestamp': 'Tue Jul 14 20:18:55 2026'}
```

foodcourt_02_gather.py

- Concept Taught: Concurrent **Execution** and **Result Aggregation**
- Key Mechanisms: **asyncio.gather()**
- Scenario: Eating with a **group of friends**, ordering **3 separate** meals from different shops simultaneously, and waiting for **all dishes to be completed** before serving them together.
- What this file teaches:
 - How to **manage** and resolve **multiple Task** objects **concurrently** using `asyncio.gather()`.
 - Understanding the "**slowest task bottleneck**" behavior, where the total elapsed time of the application will match the slowest operation (Steak: 4.0 seconds) instead of adding all operation times together.

```
Tue Jul 14 20:19:33 2026 | --- [Task 2] Practice using gather to wait for all group orders ---
Tue Jul 14 20:19:37 2026 | [Pickup] Shop: hainanese_chicken | Menu: Chicken Rice is ready!
Tue Jul 14 20:19:37 2026 | [Pickup] Shop: noodle | Menu: Wonton Noodles is ready!
Tue Jul 14 20:19:37 2026 | [Pickup] Shop: steak | Menu: Sizzling Steak is ready!
Tue Jul 14 20:19:37 2026 | Total time: 4.05 seconds (Equals to the slowest dish).
```

foodcourt_03_wait_first.py

- Concept Taught: **Racing Tasks** and **Active Resource Cleanup**
- Key Mechanisms: **asyncio.wait()**, `return_when=asyncio.FIRST_COMPLETED`, **.cancel()**
- Scenario: Being extremely **hungry, placing 3 separate orders at once**. As soon as the fastest buzzer ring, pick up that meal immediately and cancel all remaining pending orders.
- What this file teaches:
 - How to change the event loop control structure using **asyncio.wait** with the **FIRST_COMPLETED** policy.
 - How to **unpack** and **handle tasks categorized** into completed sets (done) and incomplete sets (pending).
 - Implementing network health best practices by looping through the pending set and calling `.cancel()` to drop slower connections, avoiding memory leaks and CPU waste.

```

Tue Jul 14 20:20:00 2026 | --- [Task 3] Practice using wait (FIRST_COMPLETED) ---
Tue Jul 14 20:20:00 2026 | Winner served dish: Shop: hainanese_chicken | Menu: Chicken Rice Thigh
Tue Jul 14 20:20:00 2026 | Cleaning up: Canceling 2 remaining pending orders...
Tue Jul 14 20:20:00 2026 | Total waiting time for the first dish: 0.87 seconds.

```

foodcourt_04_wait_for.py

- Concept Taught: Timeouts and Exception-Driven Control Flow
- Key Mechanisms: `asyncio.wait_for()`, `asyncio.TimeoutError`
- Scenario: Having a short lunch break. Ordering a steak (which usually takes 4s) but enforcing a hard limit of 2.0 seconds. If it exceeds the limit, skip the meal and leave the food court immediately.
- What this file teaches:
 - How to apply strict Service Level Agreements (SLAs) or time ceilings to asynchronous network operations using `asyncio.wait_for()`.
 - How to implement proper error boundaries by catching `asyncio.TimeoutError` exceptions to perform graceful fallback routines when a backend server responds too slowly.

```
--- [Task 4] Practice using wait_for to handle timeouts ---  
Tue Jul 14 20:20:28 2026 | [System] Order sent. Monitoring 2.0s timeout limit...  
Tue Jul 14 20:20:30 2026 | Timeout occurred: Steak took too long! Leaving the food court now.
```


foodcourt_05_mix_concepts.py

- Concept Taught: Advanced Integration & Nested Asynchronous Architectures
- Key Mechanisms: Combination of `create_task`, `wait_for`, and `gather`
- Scenario: Ordering noodles with a standard waiting cycle while simultaneously ordering chicken rice wrapped under a strict 1.0s timeout condition, resolving both structures inside a single collection.
- What this file teaches:
 - How to create nested architectural flows by putting an `asyncio.wait_for` wrapper directly inside an `asyncio.create_task()` execution tree.
 - Resolving complex asynchronous workflows with varying constraints inside a unified `asyncio.gather()` interface.
 - Proving that in production systems, concurrent operations, task execution, and deadline management can be combined to achieve optimal program design.

```
Tue Jul 14 20:21:06 2026 | --- [Task 5] Advanced Practice: Mixing concepts together ---  
Tue Jul 14 20:21:08 2026 | Success: All food served on time! Received 2 dishes.  
Tue Jul 14 20:21:08 2026 | Total elapsed time: 1.55 seconds.
```

Concurrency & Execution Time Grading Criteria

- **Task 1** (Create Task): Single chicken rice order takes 0.8s (Strict limit: < 1.2s).
- **Task 2** (Gather): 3 orders run concurrently. Total time must equal the slowest dish (Steak: 4.0s). Must be 3.8s – 4.5s(Fails if sequential: 6.3s).
- **Task 3** (Wait First): 3 racing orders. Chicken rice finishes fastest, so it must unblock instantly at 0.8s (Strict limit: < 1.2s).
- **Task 5** (Mix Concepts): Noodle (1.5s) + Chicken Rice with timeout (0.8s) run via gather. Total time must equal the slower task (1.5s). Bounded within 1.4s – 1.9s.